

DV-Bookshop Business Rules

Source-of-truth business requirements for testing AI Code Guardian's Business Intent Engine and LangGraph Business Agent

Application: Khit's Bookshop / DV-Bookshop

Repository: KhitMinnyo/dv-bookshop — intentionally vulnerable Flask + SQLite e-commerce training application.

Purpose: These are business requirements, not vulnerability signatures. The application should be evaluated against them; the AI agent should not invent additional requirements.

ID	Area	Required control
REQ-001	Cart / inventory	Positive quantity and available stock
REQ-002	Balance transfer	Atomic check/update; no double-spend
REQ-003	Checkout pricing	Server-calculated total is authoritative
REQ-004	Coupons	Active status and max-use limit enforced
REQ-005	Checkout payment	Sufficient balance based on authoritative total

REQ-001 — Cart Quantity and Inventory Rule

Business Requirement

A customer may add a book to the shopping cart only with a positive, valid quantity. The requested quantity must not exceed the available stock for that book. Negative, zero, or otherwise invalid quantities must not create or increase a cart item.

Expected Implementation Behavior

The implementation must validate quantity before inserting or incrementing the cart record and must respect available stock.

Violation / Compliant Conditions

Violation: arbitrary quantities are accepted or cart quantity can exceed available inventory. Compliant: quantity is validated as positive and checked against stock before the cart is changed.

Suggested Validation: Test with a normal quantity, zero/negative quantity, and a quantity greater than available stock.

REQ-002 — Balance Transfer Integrity Rule

Business Requirement

A balance transfer must be atomic. The sender must have sufficient funds, and concurrent requests must not be able to spend the same available balance more than once. The sender's balance must never become negative because of a race between checking and updating the balance.

Expected Implementation Behavior

The balance check and debit must be protected as one operation using an appropriate transaction, locking strategy, or conditional atomic update.

Violation / Compliant Conditions

Violation: concurrent requests can both pass the same balance check and then debit the account. Compliant: only transfers backed by available funds can succeed under concurrent requests.

Suggested Validation: Compare sequential and concurrent transfers from a limited balance.

REQ-003 — Authoritative Checkout Price Rule

Business Requirement

The final amount charged for an order must be calculated by the server from authoritative book prices, quantities, and applicable discounts. A client-supplied total must never determine the amount charged or deducted.

Expected Implementation Behavior

The server must recompute the order total from trusted cart/database data and use that value for payment validation, balance deduction, and the persisted order.

Violation / Compliant Conditions

Violation: a client-submitted total is trusted for payment or order creation. Compliant: client-controlled totals are ignored and the server-derived total is authoritative.

Suggested Validation: Change only the submitted checkout total while keeping the cart unchanged and verify that the charged amount remains server-derived.

REQ-004 — Coupon Usage Limit Rule

Business Requirement

A coupon may be applied only while active and while its configured maximum number of uses has not been reached. Once the limit is reached, further applications must be rejected. Usage must be accounted for when the coupon is consumed by an order.

Expected Implementation Behavior

The implementation must check active status and remaining uses and reliably update usage accounting when the coupon is consumed.

Violation / Compliant Conditions

Violation: existence/active status is checked but max_uses is not enforced or usage is not reliably consumed. Compliant: excess uses are rejected.

Suggested Validation: Use a coupon repeatedly around its configured max_uses boundary.

REQ-005 — Sufficient Funds for Authoritative Order Total Rule

Business Requirement

An order may be confirmed only when the customer's available balance is at least the authoritative final order total. The same trusted total must be used for the insufficient-funds check and balance deduction. A client-controlled amount must not make an unaffordable order appear affordable.

Expected Implementation Behavior

Payment decision, balance deduction, and persisted order total must use one server-derived final amount.

Violation / Compliant Conditions

Violation: an untrusted/client-supplied amount is used for the balance check or deduction. Compliant: the server derives the final total and rejects the order when balance is insufficient.

Suggested Validation: Create an order whose real total exceeds the balance, then vary only the client-submitted amount.

Recommended AI Code Guardian Test

Upload this PDF as the Business Rules document and scan the DV-Bookshop repository. The deterministic Business Intent Engine should attempt the mappings first. Where evidence is insufficient or ambiguous, the Business Agent should compare the original requirement with relevant implementation evidence and resolve it as COMPLIANT, VIOLATION, or PARTIAL.

Verify these UI outputs:

- The PDF is recognized as the business-rule source document.
- REQ-001 through REQ-005 are represented in the deterministic analysis.
- Ambiguous/insufficient mappings trigger Business Agent reasoning.
- The AI uses the uploaded requirement plus implementation evidence rather than inventing a new requirement.
- The final AI result contains a verdict, confidence, evidence, and reasoning.
- The AI result is marked AI_VALIDATED and reaches the Business Intent UI.

Source note: Rules were derived from the current DV-Bookshop shopping/business workflows, including cart quantity handling, checkout pricing/payment, coupon application, and balance transfer. The repository identifies the application as an intentionally vulnerable e-commerce training lab. The PDF states expected business controls without reproducing exploit payloads.

Repository: <https://github.com/KhitMinnyo/dv-bookshop>